

As you have seen, to create extension modules with SWIG, you need the C/C++ source and header files of the code you want to wrap as a Python extension. You also need the Python header files to compile the wrappers. You need to write the interface file, too, but SWIG's capabilities can balance the extra effort required.

These examples should give you enough of a boost to start with SWIG. SWIG is a very powerful tool that lets you use almost all the advanced features of C/C++ to create extension modules, and also has many powerful features of its own. Covering SWIG in detail would require a book in itself; you can explore it via the documentation provided on its home page.

Pyrex and Cython—creating extension modules with Python itself

Both Ctypes and SWIG wrap existing C/C++ code that you might have to write yourself. SWIG's interface file is extra work as well. Besides, you can't create new Python types with SWIG—and if something goes wrong, then debugging SWIG-generated C/C++ code is a daunting task. Creating new functionality and debugging it in C/C++ is complicated and tedious.

Instead of all this, here's a different approach: create Python modules by writing your extension's code in Python itself! This code is then compiled to the equivalent C code before building into a shared library.

Pyrex is a specialised language very similar to Python. It allows you to create C data types and functions in Python-like code. Cython is inspired by Pyrex; it is more feature-rich and optimised than Pyrex, is under very active development and is more frequently updated than Pyrex. Therefore, this section mainly explores Cython; you can apply working knowledge from here to Pyrex without much extra effort, should you choose to do so.

To install Pyrex, if you wish to, download the latest source tarball from its home page (see *References*) and run the following commands:

```
tar -zxvf Pyrex-version.tar.gz && cd Pyrex-version \
&& sudo python setup.py install
```

To install Cython, you again require the Python development headers (install with the command `sudo apt-get install python-dev`). Download the latest source tarball of Cython from its home page, and run:

```
tar -zxvf Cython-version.tar.gz && cd Cython-version \
&& sudo python setup.py install
```

To get a glimpse of Cython in action, compile the `primes.pyx` example file taken from the Cython official documentation with `cython primes.pyx`. To compile the generated `primes.c`, run `gcc -shared -fPIC -o primes.so primes.c -I(path of Python headers)`. To test the created extension module, run `testprimes.py`.

Unlike SWIG, it doesn't matter if you name the shared library with or without an underscore before the module name: `_primes.so` or `primes.so`. The extension module in Python works for both these names.

If you look at the source of `primes.pyx`, you will notice that we are mixing C and Python types in the Python routine to calculate prime numbers. In Cython, you declare C data types, and C `struct`, `union` or `enum` types with the `cdef` keyword. You can also declare functions with `cdef`—see below.

There are two kinds of function definitions in Cython—Python functions, defined using the familiar Python `def` statement, and C functions, defined using the `cdef` statement. The first take Python objects as parameters and return Python objects, while Cython C functions can take either Python objects or C values as parameters, and can return either Python objects or C values. The small catch is that within a Cython module, Python functions and C functions can call each other freely—but you can only call Python functions from outside the module (from interpreted Python code). So, any functions that you want to 'export' from your Cython module should be declared as Python functions using `def`—or use the hybrid `cpdef`, which can be called from both outside the module and from within, but which uses the faster C calling conventions only when being called from other Cython code.

Cython's source code compiler translates Python code to the equivalent C code that is executed within the CPython runtime environment, but at the speed of compiled C, and with the ability to call directly into C libraries. Yet, it keeps the original interface of the Python source code, making it directly usable from Python code. These characteristics enable Cython to extend the CPython interpreter with fast binary modules, and also to interface Python code with external C libraries.

The sample code was just an introduction to Cython; you can learn more about it (and Pyrex) from the documentation provided on the home pages.

To sum up, extension modules greatly enhance the functionality and power of Python. You can create very innovative Python applications using the tools we've covered in this article—being limited only by your imagination. 

References

- GNU libc documentation: <http://www.gnu.org/software/libc/manual/>
- X86 calling conventions: http://en.wikipedia.org/wiki/X86_calling_conventions
- Python library documentation on Ctypes: <http://docs.python.org/library/ctypes.html>
- SWIG home page: <http://swig.org/>
- Pyrex home page: <http://www.cosc.canterbury.ac.nz/~greg/python/Pyrex>
- Cython home page: <http://cython.org/>

By: Ankur Kumar Sharma

The author is a software developer and researcher with experience of more than eight years in various areas like kernel/hardware programming, systems development, automation tools development, R&D, etc. He is now the founder of RichNusGeeks Consulting, an open source consulting and training start-up. He likes to play classic rock music on his guitar and croon along, read self-help books, write, and explore stuff that interests him in his spare time. He blogs at <http://www.richnusgeeks.wordpress.com>.